

Đây là notebook đồng hành cho cuốn sách [Deep Learning with Python, Third Edition](#). Để dễ đọc, notebook này chỉ chứa các khối mã có thể chạy được và tiêu đề các phần, lược bỏ tất cả các nội dung khác trong sách như: các đoạn văn bản, hình vẽ và giả mã.

Nếu bạn muốn hiểu rõ những gì đang diễn ra, tôi khuyên bạn nên đọc notebook này song song với bản in hoặc bản điện tử của cuốn sách.

Nội dung của cuốn sách có sẵn trực tuyến tại deeplearningwithpython.io.

```
!pip install keras keras-hub --upgrade -q
```

```
import os
os.environ["KERAS_BACKEND"] = "jax"
```

[Hiện mã](#)

- ✓ Các thực hành tốt nhất (Best practices) trong thế giới thực
- ✓ Khai thác tối đa hiệu quả từ các mô hình của bạn
- ✓ Tối ưu hóa siêu tham số (Hyperparameter optimization)
- ✓ Sử dụng KerasTuner

```
!pip install keras-tuner -q
```

```
import keras
from keras import layers

def build_model(hp):
    units = hp.Int(name="units", min_value=16, max_value=64, step=16)
    model = keras.Sequential(
        [
            layers.Dense(units, activation="relu"),
            layers.Dense(10, activation="softmax"),
        ]
    )
    optimizer = hp.Choice(name="optimizer", values=["rmsprop", "adam"])
    model.compile(
        optimizer=optimizer,
        loss="sparse_categorical_crossentropy",
        metrics=["accuracy"],
    )
    return model
```

```
import keras_tuner as kt

class SimpleMLP(kt.HyperModel):
    def __init__(self, num_classes):
        self.num_classes = num_classes

    def build(self, hp):
        units = hp.Int(name="units", min_value=16, max_value=64, step=16)
        model = keras.Sequential(
            [
                layers.Dense(units, activation="relu"),
                layers.Dense(self.num_classes, activation="softmax"),
            ]
        )
        optimizer = hp.Choice(name="optimizer", values=["rmsprop", "adam"])
        model.compile(
            optimizer=optimizer,
            loss="sparse_categorical_crossentropy",
            metrics=["accuracy"],
        )
        return model
```

```
hypermodel = SimpleMLP(num_classes=10)
```

```
tuner = kt.BayesianOptimization(  
    build_model,  
    objective="val_accuracy",  
    max_trials=20,  
    executions_per_trial=2,  
    directory="mnist_kt_test",  
    overwrite=True,  
)
```

```
tuner.search_space_summary()
```

```
(x_train, y_train), (x_test, y_test) = keras.datasets.mnist.load_data()  
x_train = x_train.reshape((-1, 28 * 28)).astype("float32") / 255  
x_test = x_test.reshape((-1, 28 * 28)).astype("float32") / 255  
x_train_full = x_train[:]  
y_train_full = y_train[:]  
num_val_samples = 10000  
x_train, x_val = x_train[:-num_val_samples], x_train[-num_val_samples:]  
y_train, y_val = y_train[:-num_val_samples], y_train[-num_val_samples:]  
callbacks = [  
    keras.callbacks.EarlyStopping(monitor="val_loss", patience=5),  
)  
tuner.search(  
    x_train,  
    y_train,  
    batch_size=128,  
    epochs=100,  
    validation_data=(x_val, y_val),  
    callbacks=callbacks,  
    verbose=2,  
)
```

```
top_n = 4  
best_hps = tuner.get_best_hyperparameters(top_n)
```

```
def get_best_epoch(hp):  
    model = build_model(hp)  
    callbacks = [  
        keras.callbacks.EarlyStopping(  
            monitor="val_loss", mode="min", patience=10  
        )  
    ]  
    history = model.fit(  
        x_train,  
        y_train,  
        validation_data=(x_val, y_val),  
        epochs=100,  
        batch_size=128,  
        callbacks=callbacks,  
    )  
    val_loss_per_epoch = history.history["val_loss"]  
    best_epoch = val_loss_per_epoch.index(min(val_loss_per_epoch)) + 1  
    print(f"Best epoch: {best_epoch}")  
    return best_epoch
```

```
def get_best_trained_model(hp):  
    best_epoch = get_best_epoch(hp)  
    model = build_model(hp)  
    model.fit(  
        x_train_full, y_train_full, batch_size=128, epochs=int(best_epoch * 1.2)  
    )  
    return model  
  
best_models = []  
for hp in best_hps:  
    model = get_best_trained_model(hp)  
    model.evaluate(x_test, y_test)  
    best_models.append(model)
```

```
best_models = tuner.get_best_models(top_n)
```

Nghệ thuật xây dựng không gian tìm kiếm phù hợp

Tương lai của việc tinh chỉnh siêu tham số: Học máy tự động (AutoML)

Tập hợp mô hình (Model ensembling)

✓ Mở rộng quy mô huấn luyện mô hình với nhiều thiết bị

✓ Huấn luyện trên nhiều GPU

Song song hóa dữ liệu (Data parallelism): Sao chép mô hình trên mỗi GPU

Song song hóa mô hình (Model parallelism): Chia nhỏ mô hình trên nhiều GPU

✓ Thực hành huấn luyện phân tán

Cách tiếp cận với hai hoặc nhiều GPU

Sử dụng song song hóa dữ liệu với JAX

✓ Sử dụng song song hóa mô hình với JAX

API DeviceMesh

API LayoutMap

✓ Huấn luyện trên TPU

Sử dụng kỹ thuật hợp nhất bước (step fusing) để cải thiện hiệu suất sử dụng TPU

✓ Tăng tốc huấn luyện và suy luận với tính toán độ chính xác thấp (lower-precision)

Hiệu về độ chính xác số dấu phẩy động (floating-point precision)

Suy luận với Float16

Huấn luyện với độ chính xác hỗn hợp (Mixed-precision training)

Sử dụng thang đo mất mát (loss scaling) với mixed precision

Vượt xa hơn mixed precision: huấn luyện với float8

✓ Tăng tốc suy luận với lượng tử hóa (quantization)

```
from keras import ops
```

```
x = ops.array([[0.1, 0.9], [1.2, -0.8]])  
kernel = ops.array([[-0.1, -2.2], [1.1, 0.7]])
```

```
def abs_max_quantize(value):  
    abs_max = ops.max(ops.abs(value), keepdims=True)  
    scale = ops.divide(127, abs_max + 1e-7)  
    scaled_value = value * scale
```

```
scaled_value = ops.clip(ops.round(scaled_value), -127, 127)
scaled_value = ops.cast(scaled_value, dtype="int8")
return scaled_value, scale

int_x, x_scale = abs_max_quantize(x)
int_kernel, kernel_scale = abs_max_quantize(kernel)
```

```
int_y = ops.matmul(int_x, int_kernel)
y = ops.cast(int_y, dtype="float32") / (x_scale * kernel_scale)
```

```
y
```

```
ops.matmul(x, kernel)
```